

Операционные системы

Лекция 9

Тупики

План лекции

- Понятие тупика
- Условия возникновения тупиков
- Подходы к разрешению проблемы тупиков
- Дисциплины предотвращения тупиков
- Алгоритм банкира для обхода тупиков
- Граф запросов и распределения ресурсов в алгоритмах обнаружения тупиков

Тупики и подходы к их разрешению

Как мы уже говорили в компьютерных системах множество ресурсов, которые одновременно могут использоваться только одним процессом

Поэтому все операционные системы способны временно предоставлять процессу исключительные права доступа к конкретным ресурсам (в общем-то в этом и есть суть синхронизации)

Однако при работе многих приложений процессу может быть нужен исключительный доступ не к одному, а сразу к нескольким ресурсам

Тупики и подходы к их разрешению

Предположим, к примеру, что каждый из двух процессов захотел записать отсканированный документ на Blu-ray-диск. Процесс А запрашивает разрешение на использование сканера и получает его

Процесс В запрограммирован по-другому: сначала он запрашивает разрешение на использование пишущего привода Blu-ray-дисков и также получает это разрешение

Теперь А запрашивает разрешение на использование пишущего привода Blu-ray-дисков, но запрос отклоняется до тех пор, пока это устройство не будет освобождено процессом В. К сожалению, вместо того чтобы освободить привод, В запрашивает разрешение на использование сканера

Тупики и подходы к их разрешению

И в этот момент оба процесса оказываются заблокированными навсегда

И как мы уже упоминали в лекции по синхронизации, такая ситуация называется **взаимоблокировкой (deadlock)** или **тупиковой ситуацией (тупиком)**

Взаимоблокировки могут случаться и между машинами, например, если для нескольких устройств рассматривать тот же принтер в качестве общего ресурса

Взаимоблокировки могут возникать и при массе других обстоятельств

Понятие ресурсов

Основная часть взаимоблокировок связана с ресурсами, к которым некоторым процессам были предоставлены исключительные права доступа. К их числу относятся устройства, записи данных, файлы и т. д.

Чтобы придать рассмотрению взаимоблокировок как можно более универсальный характер, мы будем называть объекты, к которым предоставляется доступ, **ресурсами**

Ресурсами могут быть аппаратные устройства (например, привод Blu-ray-дисков) или какая-то часть информации (например, запись базы данных)

Понятие ресурсов

Обычно у компьютера может быть множество ресурсов, которые могут быть предоставлены процессу. Некоторые ресурсы могут быть доступны в нескольких идентичных экземплярах, например три привода Blu-ray-дисков

Когда доступны несколько копий ресурса, то для удовлетворения любого запроса на ресурс может быть использован один из них

Короче говоря, под ресурсом понимается все, что должно предоставляться, использоваться и через некоторое время высвобождаться, поскольку в один и тот же момент времени может использоваться только одним процессом

Понятие ресурсов

Ресурсы бывают двух видов: выгружаемые и невыгружаемые

К **выгружаемым** относятся такие ресурсы, которые могут быть безболезненно отображены у процесса, который ими обладает

Примером такого ресурса может послужить память.

Рассмотрим систему, имеющую 1 Гбайт пользовательской памяти, один принтер и два процесса по 1 Гбайт, каждый из которых хочет что-то вывести на печать. Процесс А запрашивает и получает принтер, а затем начинает вычислять значение, предназначенное для вывода на печать. Но до завершения вычисления истекает выделенный ему квант времени, и он выгружается на диск

Понятие ресурсов

Теперь запускается процесс В, безуспешно, как оказывается, пытаясь завладеть принтером. Потенциально возникает ситуация взаимоблокировки, поскольку у процесса А есть принтер, а у процесса В – память и ни один из них не может продолжить свою работу без ресурса, удерживаемого другим процессом

К счастью, есть возможность отобрать память у процесса В, выгрузив этот процесс на диск, и загрузить оттуда процесс А

Теперь А может возобновить свою работу, выполнить распечатку и высвободить принтер. И никакой взаимоблокировки не возникнет

Понятие ресурсов

А вот **невыгружаемый** ресурс нельзя отобрать у его текущего владельца, не вызвав потенциально сбоя в вычислениях

Если у процесса, который уже приступил к записи на Blu-ray-диск, внезапно отобрать пишущий привод и отдать его другому процессу, это приведет к порче Blu-ray-диска. Пишущие приводы Blu-ray-дисков нельзя отобрать в произвольный момент

Выгружаемость ресурса зависит от контекста

Как правило, во взаимоблокировках фигурируют невыгружаемые ресурсы. Обычно потенциальные взаимоблокировки с участием выгружаемых ресурсов могут быть устранены путем перераспределения ресурсов от одного процесса к другому

Понятие ресурсов

В наиболее общем виде при использовании ресурса происходит следующая последовательность событий:

1. Запрос ресурса
2. Использование ресурса
3. Высвобождение ресурса

Если во время запроса ресурс недоступен, запрашивающий процесс вынужден перейти к ожиданию. В некоторых операционных системах при отказе в выделении запрошенного ресурса процесс автоматически блокируется, а когда ресурс становится доступен – возобновляется

В других системах отказ в выделении запрашиваемого ресурса сопровождается кодом ошибки, и принятие решения о том, что следует делать, немного подождать или попытаться снова получить ресурс, возлагается на вызывающий процесс

Понятие ресурсов

Для некоторых видов ресурсов, таких как записи в базе данных, управление использованием ресурсов зависит от самих пользовательских процессов, а не от системы

Один из способов, позволяющих ввести пользовательское управление ресурсами, заключается в присоединении семафора к каждому из ресурсов

Все эти семафоры получают исходное значение, равное 1. С таким же успехом могут использоваться и мьютексы

Перечисленные ранее три этапа затем воплощаются в применении к семафору вызова *down* для получения ресурса, использование ресурса и, в завершение, применение вызова *up* при высвобождении ресурса

Понятие ресурсов

... ..

```
typedef int semaphore;  
semaphore resource_1;
```

```
void process_A(void) {  
    down(&resource_1);  
    use_resource_1( );  
    up(&resource_1);  
}
```

a

```
typedef int semaphore;  
semaphore resource_1;  
semaphore resource_2;
```

```
void process_A(void) {  
    down(&resource_1);  
    down(&resource_2);  
    use_both_resources( );  
    up(&resource_2);  
    up(&resource_1);  
}
```

б

Понятие ресурсов

Пока все идет хорошо. Пока речь идет только об одном процессе, все работает нормально. Конечно, когда используется только один процесс, нет нужды в формальном получении ресурсов, поскольку нет соперничества за обладание ими

Теперь рассмотрим ситуацию с двумя процессами – А и В – и двумя ресурсами

Рассмотрим два сценария:

а – оба процесса запрашивают ресурсы в одном и том же порядке

б – запрашивают ресурсы в разном порядке

Разница может показаться несущественной, но это не так

Понятие ресурсов

```
typedef int semaphore;
    semaphore resource_1;
    semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

void process_B(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}
```

a

```
semaphore resource_1;
semaphore resource_2;

void process_A(void) {
    down(&resource_1);
    down(&resource_2);
    use_both_resources( );
    up(&resource_2);
    up(&resource_1);
}

void process_B(void){
    down(&resource_2);
    down(&resource_1);
    use_both_resources( );
    up(&resource_1);
    up(&resource_2);
}
```

б

Понятие ресурсов

В случае а – один из процессов запрашивает первый ресурс раньше, чем это делает второй процесс. Затем этот же процесс успешно получает второй ресурс и выполняет свою работу. Если второй процесс попытается получить ресурс 1 до его высвобождения, то он будет просто заблокирован до тех пор, пока ресурс не станет доступен

В случае б – один из процессов получит оба ресурса и надежно заблокирует другой процесс до тех пор, пока не сделает свою работу. Но может случиться и так, что процесс А получит ресурс 1, а процесс В получит ресурс 2. Каждый из них теперь будет заблокирован при попытке получения второго ресурса. Ни один из процессов не возобновит свою работу

Плохо то, что возникнет ситуация взаимоблокировки

Понятие ресурсов

Мы видим, что происходит из-за небольшой разницы в стиле программирования: в зависимости от того, какой из ресурсов будет получен первым, программа либо работает, либо дает трудноопределимый сбой

Поскольку взаимоблокировки могут возникать столь просто, для борьбы с ними были проведены обширные исследования. Поэтому стоит рассмотреть взаимоблокировки и средства борьбы с ними

Тупики и подходы к их разрешению

Формально определим взаимоблокировку так:

Взаимоблокировка в группе процессов возникает в том случае, если каждый процесс из этой группы ожидает события, наступление которого зависит исключительно от другого процесса из этой же группы

В большинстве случаев событием, наступления которого ожидает каждый процесс, является высвобождение какого-либо ресурса, которым на данный момент владеет другой участник группы

Каждый процесс из группы, попавшей в ситуацию взаимоблокировки, ожидает ресурса, которым обладает другой процесс из этой же группы. Ни один из процессов не может работать, ни один из них не может высвободить какой-либо ресурс, и ни один из них не может возобновить свою работу

Этот результат сохраняется для любого типа ресурсов, включая аппаратные и программные ресурсы. Этот вид взаимоблокировки называется **ресурсной взаимоблокировкой**

Тупики и подходы к их разрешению

И вновь причиной взаимоблокировки является ошибка при разработке программы. Такие ошибки обычно довольно таинственны и трудноуловимы. Кроме того, может оказаться, что взаимоблокировку вызывает только достаточно редкая комбинация событий, и тогда до того, как ошибка проявит себя, программа может быть в эксплуатации многие годы

Единой эффективной **стратегии** для работы со всеми типами взаимоблокировок **нет**

Основные подходы к решению этой проблемы следующие:

- Игнорирование проблемы
- Предотвращение взаимоблокировок
- Устранение взаимоблокировок
- Обнаружение взаимоблокировок

Тупики и подходы к их разрешению

Игнорирование проблемы. Может быть, если вы проигнорируете ее, она проигнорирует вас

Предотвращение взаимоблокировок. Не позволять осуществиться одному из необходимых условий возникновения взаимоблокировки

Устранение взаимоблокировок. Не выполнять запрос ресурса, если его выделение может привести к взаимоблокировке.

Обнаружение взаимоблокировок. По возможности выполнять запросы ресурсов, но периодически проверять наличие взаимоблокировки и принимать меры для восстановления

Мы изучим все перечисленные подходы к решению проблемы, но сначала рассмотрим **условия возникновения взаимоблокировок**

УСЛОВИЯ ВОЗНИКНОВЕНИЯ

Эдвард Коффман показал, что для возникновения ресурсных взаимоблокировок должны выполняться четыре условия:

- 1. Условие взаимного исключения.** Каждый ресурс либо выделен в данный момент только одному процессу, либо доступен
- 2. Условие удержания и ожидания.** Процессы, удерживающие в данный момент ранее выделенные им ресурсы, могут запрашивать новые ресурсы
- 3. Условие невыгружаемости.** Ранее выделенные ресурсы не могут быть принудительно отобраны у процесса. Они должны быть явным образом высвобождены тем процессом, который их удерживает

УСЛОВИЯ ВОЗНИКНОВЕНИЯ

4. Условие циклического ожидания. Должна существовать кольцевая последовательность из двух и более процессов, каждый из которых ожидает высвобождения ресурса, удерживаемого следующим членом последовательности

Для возникновения ресурсной взаимоблокировки **ДОЛЖНЫ соблюдаться все четыре условия.** Если одно из них не соблюдается, ресурсная взаимоблокировка невозможна

Следует заметить, что каждое условие относится к той политике, которой придерживается или не придерживается система. Может ли данный ресурс быть выделен одновременно более чем одному процессу? Может ли процесс удерживать ресурс и запрашивать другой ресурс? Может ли ресурс быть отобран? Может ли получиться циклическое ожидание?

Графы ресурсов

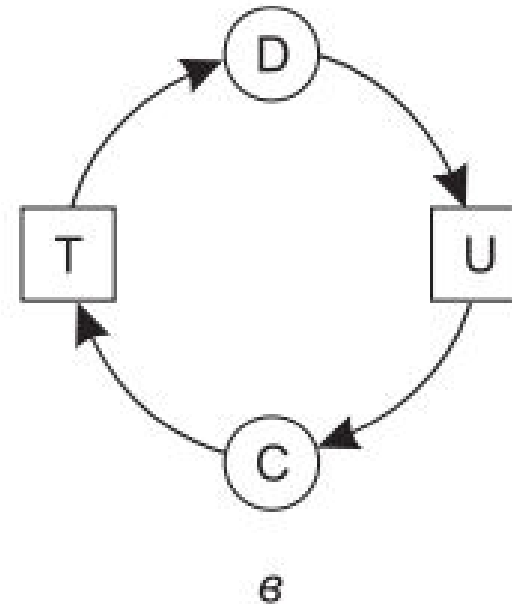
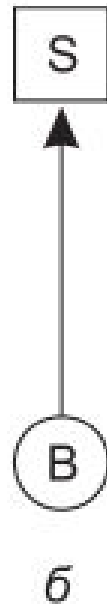
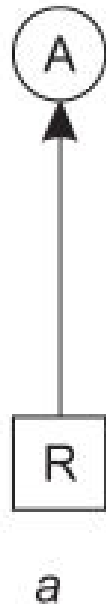
Ричард Холт показал, как эти четыре условия могут быть смоделированы с использованием направленных графов

У графов имеется два вида узлов: **процессы**, показанные **окружностями**, и **ресурсы**, показанные **квадратами**

Направленное ребро, которое следует от узла ресурса (квадрата) к узлу процесса (окружности), означает, что этот ресурс был ранее запрошен, получен и на данный момент удерживается этим процессом (а)

Направленное ребро, идущее от процесса к ресурсу, означает, что процесс в данное время заблокирован в ожидании высвобождения этого ресурса (б)

Графы ресурсов



Графы ресурсов

На графе (в) мы наблюдаем взаимоблокировку:

- процесс С ожидает высвобождения ресурса Т, который в данный момент удерживается процессом D
- процесс D не собирается высвободить ресурс Т, поскольку он ожидает высвобождения ресурса U, удерживаемого процессом С

Оба процесса находятся в состоянии вечного ожидания

Циклическая структура графа означает, что мы имеем дело с взаимоблокировкой, включившей процессы и ресурсы в цикл (предполагается, что в системе есть только один ресурс каждого типа)

В данном примере получился следующий цикл: С–Т–D–U–С

Графы ресурсов

Рассмотрим пример того, как могут быть использованы графы ресурсов

Представим, что есть 3 процесса, А, В и С, и 3 ресурса, R, S и T

Последовательности действий по запросу и высвобождению ресурсов, осуществляемые этими тремя процессами:

А	В	С
Запросить R	Запросить S	Запросить T
Запросить S	Запросить T	Запросить R
Освободить R	Освободить S	Освободить T
Освободить S	Освободить T	Освободить R
<i>а</i>	<i>б</i>	<i>в</i>

Графы ресурсов

Операционная система может в любое время запустить любой незаблокированный процесс, то есть она может принять решение запустить процесс А и дожидаться, пока он не завершит всю свою работу, затем запустить процесс В и довести его работу до завершения и, наконец, запустить процесс С (*)

Теперь предположим, что процессы занимают как ввод-выводом, так и вычислениями, поэтому наиболее разумным алгоритмом планирования их работы является циклический алгоритм

Графы ресурсов

Запросы ресурсов могут происходить в том порядке:

1. А запрашивает R
 2. В запрашивает S
 3. С запрашивает T
 4. А запрашивает S
 5. В запрашивает T
 6. С запрашивает R
- Взаимоблокировка

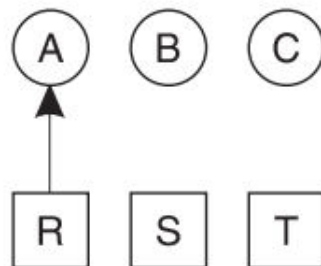
2

Если эти шесть запросов выполняются именно в этом порядке, им соответствующую шесть результирующих ресурсных графов

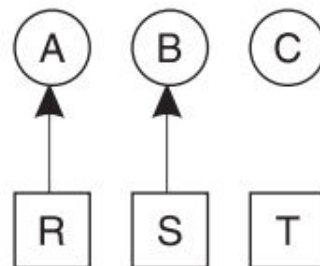
Графы ресурсов

1. A запрашивает R
 2. B запрашивает S
 3. C запрашивает T
 4. A запрашивает S
 5. B запрашивает T
 6. C запрашивает R
- Взаимоблокировка

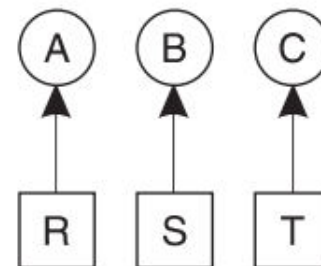
а



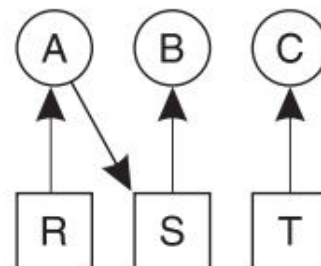
б



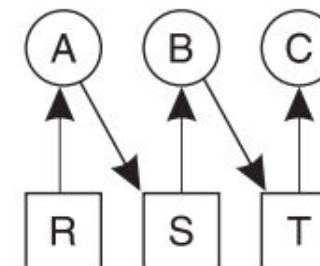
в



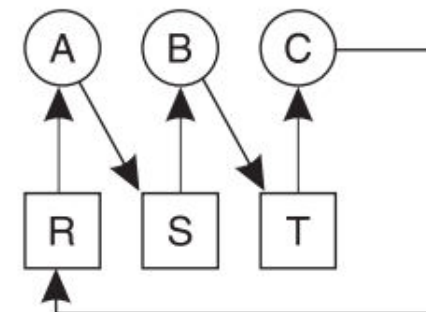
г



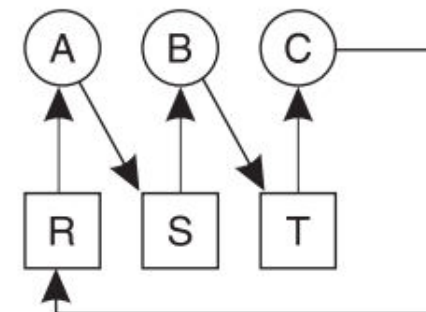
д



е



ж



Графы ресурсов

Но как уже ранее упоминалось, от операционной системы не требовалось запускать процессы в каком-то определенном порядке

В частности, если удовлетворение конкретного запроса может привести к взаимоблокировке, операционная система может просто приостановить процесс, не удовлетворяя его запрос (то есть просто не планируя работу процесса) до тех пор, пока это не будет безопасно

Если операционная система знает о грядущей взаимоблокировке, она может приостановить процесс В, вместо того чтобы выделить ему ресурс S

Графы ресурсов

Запуская только процессы А и С, мы получим такую последовательность действий по запросу и высвобождению ресурсов:

1. А запрашивает R
2. С запрашивает T
3. А запрашивает S
4. С запрашивает R
5. А освобождает R
6. А освобождает S

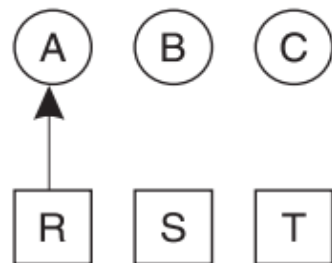
Нет взаимоблокировки

Л

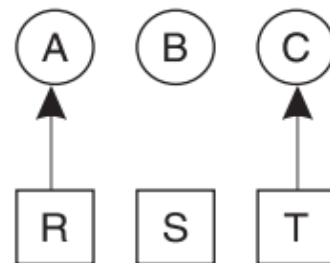
Эта последовательность отображается следующими ресурсными графами и не приводит к взаимоблокировке

Графы ресурсов

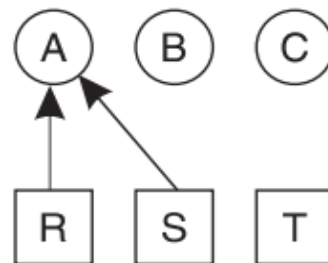
1. A запрашивает R
 2. C запрашивает T
 3. A запрашивает S
 4. C запрашивает R
 5. A освобождает R
 6. A освобождает S
- Нет взаимоблокировки



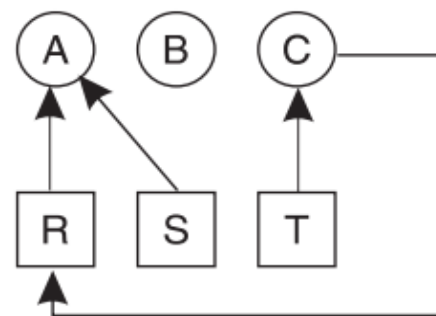
л



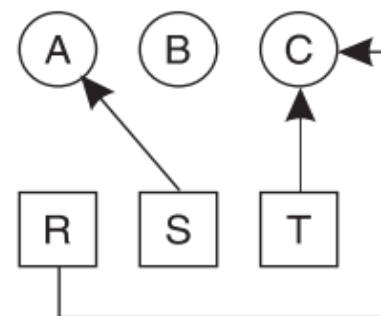
м



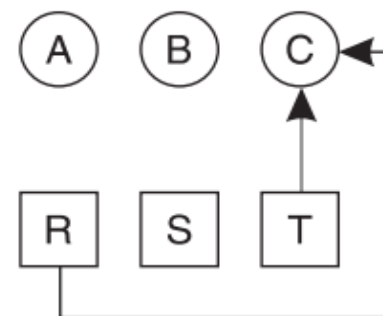
н



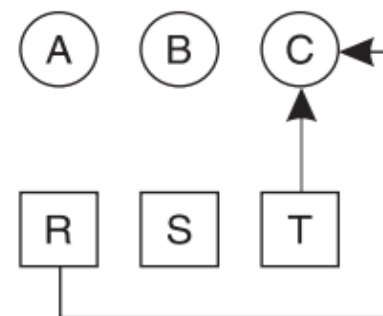
о



п



р



с

Графы ресурсов

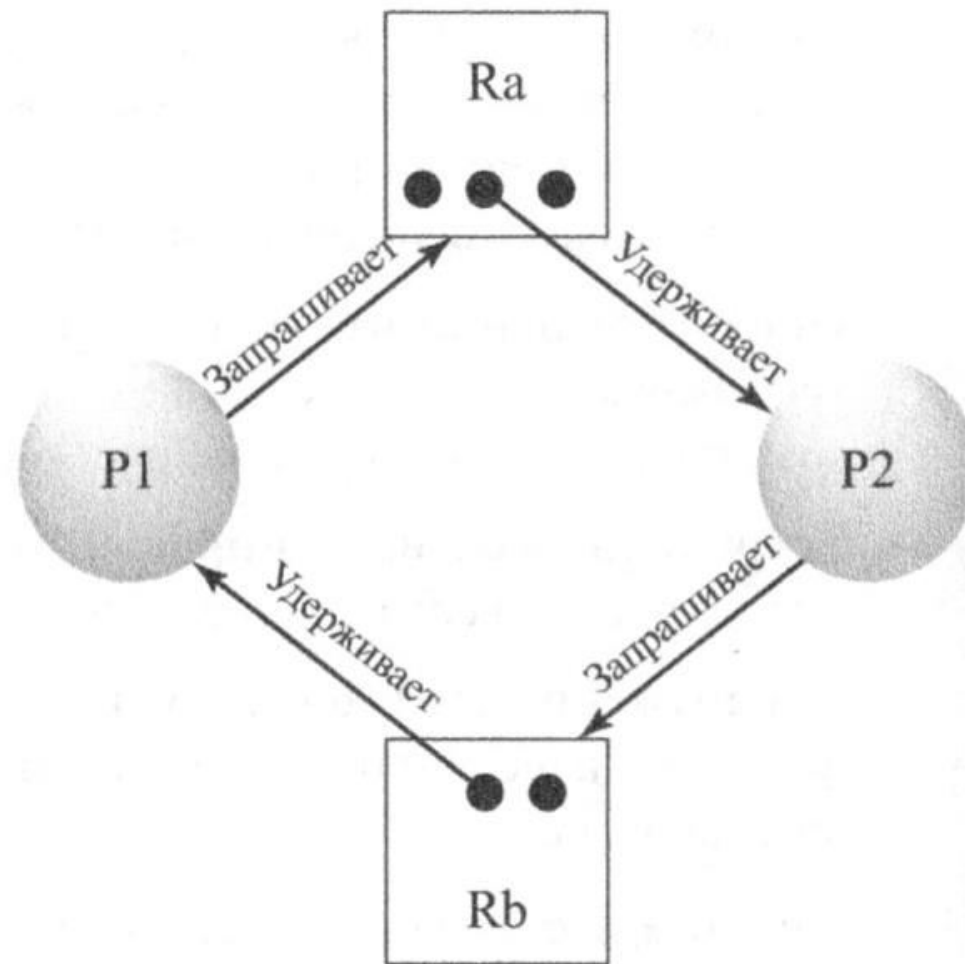
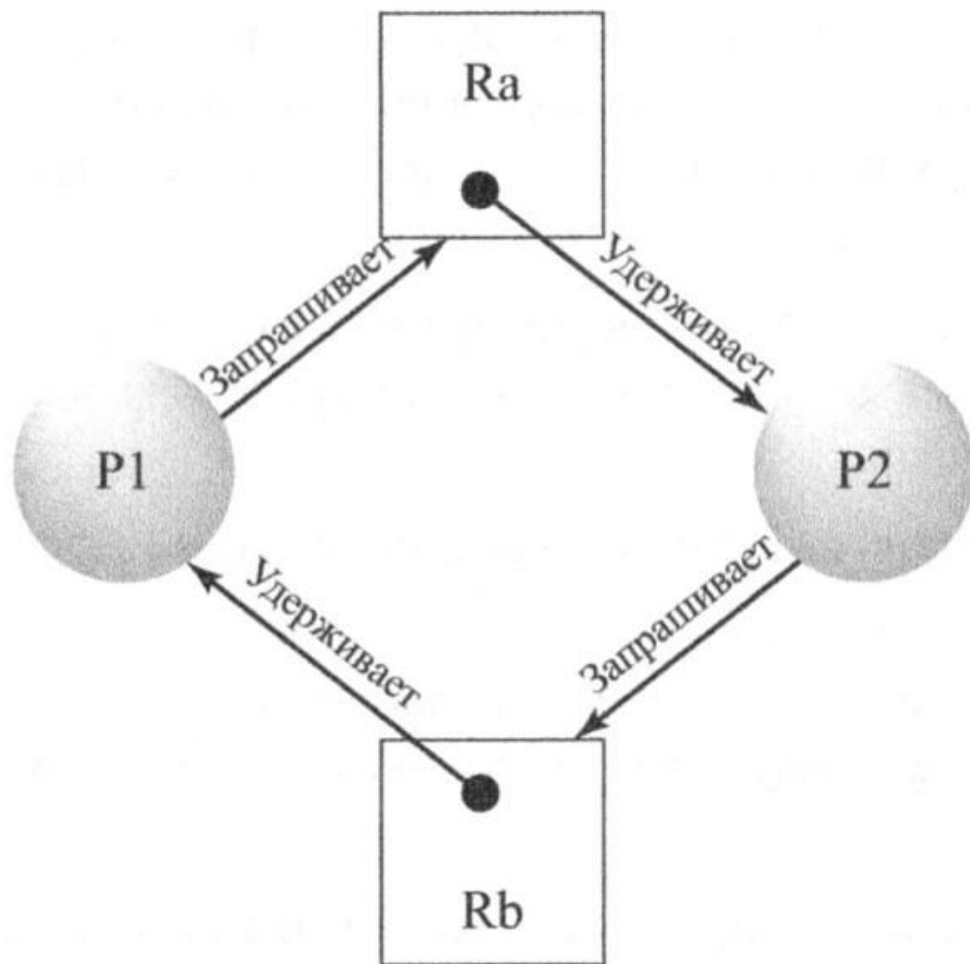
Ресурсные графы являются инструментом, позволяющим понять, приводит ли данная последовательность запросов/возвратов ресурсов к взаимоблокировке

Мы всего лишь шаг за шагом осуществляем запросы и возвраты ресурсов и после каждого шага проверяем граф на наличие каких-либо циклов

Если образуется цикл, значит, возникла взаимоблокировка, а если нет, значит, нет и взаимоблокировки

Хотя мы рассматривали граф ресурсов, составленный для случая использования по одному ресурсу каждого типа, ресурсные графы могут быть применены также для обработки нескольких ресурсов одного и того же типа

Графы ресурсов



Страусиный алгоритм

Самым простым подходом к решению проблемы является **«страусиный алгоритм»**: спрячьте голову в песок и сделайте вид, что проблема отсутствует

Люди реагируют на эту стратегию по-разному:

- Математики считают ее неприемлемой и говорят, что взаимоблокировки следует предотвращать любой ценой
- Инженеры спрашивают, как часто ожидается возникновение проблемы, как часто система дает сбой по другим причинам и насколько серьезны последствия взаимоблокировки

Если взаимоблокировка возникает в среднем один раз в пять лет, а система раз в неделю сбоит из-за технических отказов и дефектов операционной системы, большинство инженеров не захотят платить за избавление от взаимоблокировок существенным снижением производительности или удобства использования

Обнаружение и восстановление

Вторая по счету – **технология обнаружения и восстановления**. При ее использовании система не пытается предотвращать взаимоблокировки. Она позволяет им произойти, пытается обнаружить момент их возникновения, а затем предпринимает некоторые действия по восстановлению работоспособности

Рассмотрим следующие моменты:

- Обнаружение взаимоблокировки при использовании одного ресурса каждого типа
- Обнаружение взаимоблокировки при использовании нескольких ресурсов каждого типа
- Выход из взаимоблокировки

Обнаружение и восстановление

Начнем с самого простого случая, когда используется только один ресурс каждого типа. У системы может быть один сканер, один пишущий привод Blu-ray-дисков, один плоттер и один накопитель на магнитной ленте, но только по одному экземпляру каждого класса ресурсов

Иными словами, мы временно исключаем системы, имеющие два принтера

Для такой системы можно построить ресурсный граф. Если этот граф содержит один и более циклов, значит, мы имеем дело с взаимоблокировкой. Любой процесс, являющийся частью цикла, заблокирован намертво. Если циклов нет, значит, система не находится в состоянии взаимоблокировки

Обнаружение и восстановление

В качестве примера более сложной системы по сравнению с ранее рассмотренными, возьмем систему с семью процессами от А до G и шестью ресурсами от R до W. Каждый ресурс находится в состоянии текущей занятости, и на каждый ресурс в данный момент поступил запрос:

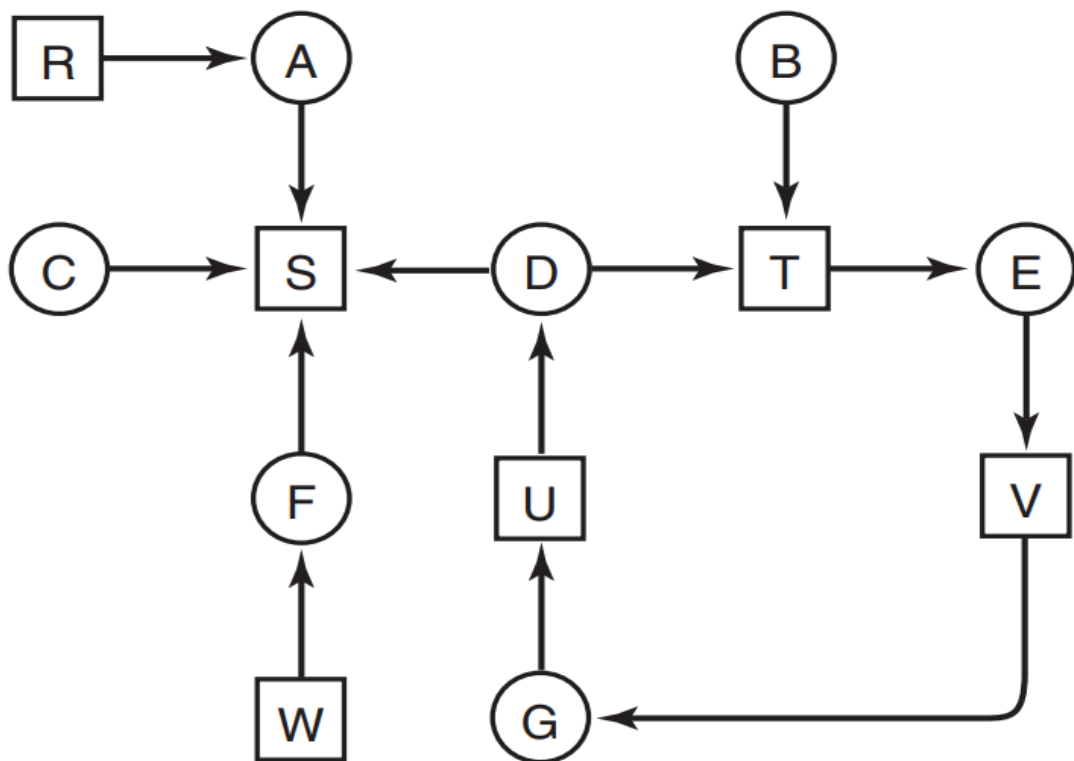
1. Процесс А удерживает R и хочет получить S
2. Процесс В не удерживает никаких ресурсов, но хочет получить T
3. Процесс С не удерживает никаких ресурсов, но хочет получить S
4. Процесс D удерживает U и хочет получить S и T
5. Процесс Е удерживает T и хочет получить V
6. Процесс F удерживает W и хочет получить S
7. Процесс G удерживает V и хочет получить U

Обнаружение и восстановление

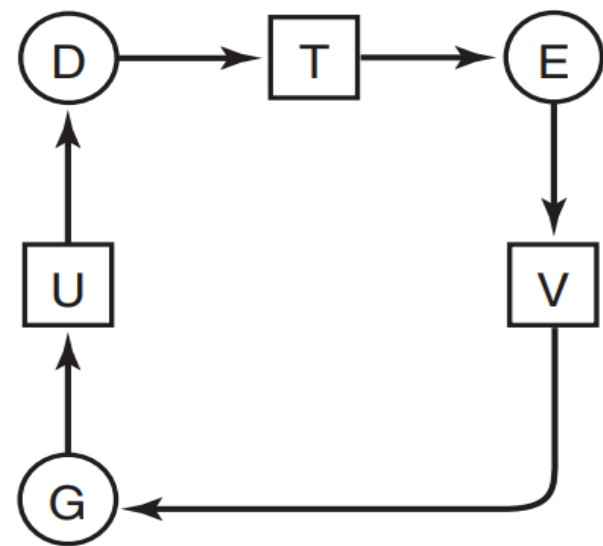
Возникает следующий вопрос: «Находится ли эта система в состоянии взаимоблокировки, и если находится, то какие процессы вовлечены в это состояние?»

Чтобы ответить на этот вопрос, можно построить граф ресурсов

Обнаружение и восстановление



(a)



(b)

Обнаружение и восстановление

Этот граф содержит один цикл (b), который можно обнаружить визуально

Из цикла видно, что процессы D, E и G вовлечены во взаимоблокировку

Процессы A, C и F не находятся в состоянии взаимоблокировки, поскольку ресурс S может быть выделен любому из них, который затем закончит свою работу и вернет ресурс. Затем оставшиеся два процесса смогут взять его по очереди и также завершить свою работу

(Заметьте, чтобы сделать этот пример немного интереснее, мы позволили процессам, а именно процессу D, запрашивать одновременно два ресурса)

Обнаружение и восстановление

Хотя визуально выделить взаимоблокировку из простого графа относительно нетрудно, для использования в настоящих системах нужен формальный алгоритм обнаружения взаимоблокировок. Известно множество алгоритмов для обнаружения циклов в направленных графах

Далее будет приведен простой алгоритм, проверяющий граф и прекращающий свою работу либо при обнаружении цикла, либо при обнаружении отсутствия циклов

В нем используется одна динамическая структура данных, L , представляющая собой список узлов, а также список ребер. В процессе работы алгоритма ребра будут помечаться для обозначения того, что они уже были проверены, чтобы предотвратить повторные проверки

Обнаружение и восстановление

Действие алгоритма основано на выполнении следующих шагов:

1. Для каждого узла N , имеющегося в графе, выполняются следующие пять шагов, использующих узел N в качестве начального
2. Инициализируется (очищается) список L , а со всех ребер снимаются пометки
3. Текущий узел добавляется к концу списка L , и проводится проверка, не появится ли этот узел в списке L дважды. Если это произойдет, значит, граф содержит цикл (отображенный в списке L), и алгоритм прекращает работу
4. Для заданного узла определяется, нет ли каких-нибудь отходящих от него непомеченных ребер. Если такие ребра есть, осуществляется переход к шагу 5, если их нет, осуществляется переход к шагу 6

Обнаружение и восстановление

5. Произвольно выбирается и помечается непомеченное отходящее от узла ребро. Затем по нему осуществляется переход к новому текущему узлу, и алгоритм возвращается к шагу 3

6. Если этот узел является первоначальным узлом, значит, граф не содержит никаких циклов, и алгоритм завершает свою работу. В противном случае алгоритм зашел в тупик. Этот узел удаляется, и алгоритм возвращается к предыдущему узлу, то есть к тому узлу, который был текущим перед только что удаленным узлом. Данный узел делается текущим, и осуществляется переход к шагу 3

Обнаружение и восстановление

Когда в системе существует несколько экземпляров каких-нибудь ресурсов, для обнаружения взаимоблокировки необходим другой подход

Сейчас будет представлен алгоритм, основанный на использовании матриц и предназначенный для обнаружения взаимоблокировки при работе n процессов, от P_1 до P_n .

Пусть m – это число классов ресурсов, E_1 – количество ресурсов класса 1, E_2 – количество ресурсов класса 2, а в общем E_i – количество ресурсов класса i (где $1 \leq i \leq m$)

E – это **вектор существующих ресурсов**. Он передает общее количество имеющихся в наличии экземпляров каждого ресурса

Например, если класс 1 представляет собой накопители на магнитных лентах, то $E_1 = 2$ означает, что в системе есть два таких накопителя

Обнаружение и восстановление

В любой момент времени какие-то ресурсы могут быть выделены и недоступны

Пусть A будет **вектором доступных ресурсов**, где A_i дает количество экземпляров ресурса i , доступных на данный момент (то есть не выделенных). Если оба накопителя на магнитной ленте уже выделены, A_1 будет равно 0

Теперь нам нужны два массива: C – **матрица текущего распределения** и R – **матрица запросов**. i -я строка в матрице C говорит о том, сколько экземпляров каждого класса ресурсов в данный момент удерживает процесс P_i . Таким образом, C_{ij} – это количество экземпляров ресурса j , которое удерживается процессом i . По аналогии с этим R_{ij} – это количество экземпляров ресурса j , которое хочет получить процесс P_i

Обнаружение и восстановление

Resources in existence
($E_1, E_2, E_3, \dots, E_m$)

Resources available
($A_1, A_2, A_3, \dots, A_m$)

Current allocation matrix

$$\begin{bmatrix} C_{11} & C_{12} & C_{13} & \dots & C_{1m} \\ C_{21} & C_{22} & C_{23} & \dots & C_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ C_{n1} & C_{n2} & C_{n3} & \dots & C_{nm} \end{bmatrix}$$

Row n is current allocation
to process n

Request matrix

$$\begin{bmatrix} R_{11} & R_{12} & R_{13} & \dots & R_{1m} \\ R_{21} & R_{22} & R_{23} & \dots & R_{2m} \\ \vdots & \vdots & \vdots & & \vdots \\ R_{n1} & R_{n2} & R_{n3} & \dots & R_{nm} \end{bmatrix}$$

Row 2 is what process 2 needs

Обнаружение и восстановление

Для этих четырех структур данных сохраняется одно важное соотношение. А именно – каждый ресурс является либо выделенным, либо доступным. Это наблюдение означает, что:

$$\sum_{i=1}^n C_{ij} + A_j = E_j.$$

Иными словами, если сложить все уже выделенные экземпляры ресурса j и к ним прибавить все еще доступные экземпляры, в результате получится количество существующих экземпляров ресурса этого класса

Обнаружение и восстановление

Алгоритм обнаружения взаимоблокировок основан на сравнении векторов

Определим, что для двух векторов A и B соотношение $A \leq B$ означает, что каждый элемент вектора A меньше или равен соответствующему элементу вектора B .

Математически это можно записать так: $A \leq B$ тогда и только тогда, когда $A_i \leq B_i$ для $1 \leq i \leq m$

Каждый процесс изначально объявляется немаркированным. По мере работы процессы будут помечаться, показывая, что они способны завершить свою работу и не участвуют во взаимоблокировке

Обнаружение и восстановление

Когда алгоритм завершает свою работу, любой непомеченный процесс считается участвующим во взаимоблокировке. При работе этого алгоритма предполагается наихудший из возможных сценариев развития событий: все процессы удерживают все полученные ресурсы до тех пор, пока не закончат свою работу

Теперь алгоритм обнаружения взаимного исключения можно изложить в следующей последовательности:

1. Поиск непомеченного процесса, P_i , для которого i -я строка матрицы R меньше или равна A
2. Если такой процесс найден, прибавление к A i -й строки матрицы S , установка метки на процесс и возвращение к шагу 1
3. Если такого процесса нет, алгоритм завершает работу

Обнаружение и восстановление

На первом шаге алгоритм ищет процесс, который может доработать до конца. Такой процесс характеризуется тем, что все его запросы на ресурсы могут быть удовлетворены за счет текущих доступных ресурсов

Тогда выбранный процесс доработает до конца, после чего вернет все удерживаемые им ресурсы в фонд доступных ресурсов. Затем этот процесс помечается завершенным

Если в итоге окажется, что все процессы могут доработать до конца, значит, ни один из них не участвует во взаимоблокировке

Если часть процессов никогда не сможет доработать до конца, значит, они находятся в состоянии взаимоблокировки

Хотя алгоритм не является детерминированным (поскольку он может запускать процессы в любом возможном порядке), результат всегда одинаков

Обнаружение и восстановление

Рассмотрим следующий пример работы алгоритма обнаружения взаимоблокировок:

$$E = \begin{matrix} & \begin{matrix} \text{Накопители на} \\ \text{магнитной ленте} \\ \text{Плоттеры} \\ \text{Сканеры} \\ \text{Компакт-диски} \end{matrix} \\ \begin{matrix} 4 \\ 2 \\ 3 \\ 1 \end{matrix} \end{matrix}$$

$$A = \begin{matrix} & \begin{matrix} \text{Накопители на} \\ \text{магнитной ленте} \\ \text{Плоттеры} \\ \text{Сканеры} \\ \text{Компакт-диски} \end{matrix} \\ \begin{matrix} 2 \\ 1 \\ 0 \\ 0 \end{matrix} \end{matrix}$$

Матрица текущего
распределения

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Матрица запросов

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Обнаружение и восстановление

У нас есть три процесса и четыре класса ресурсов, которые мы произвольно обозначили как накопители на магнитной ленте, плоттеры, сканер и привод Blu-ray-дисков

Процесс 1 удерживает один сканер. Процесс 2 удерживает два ленточных привода и один привод Blu-ray-дисков. Процесс 3 удерживает плоттер и два сканера. Каждый процесс нуждается в дополнительных ресурсах, что отображено в матрице R

Во время работы алгоритма обнаружения взаимоблокировок осуществляется поиск процесса, чей запрос на ресурс может быть удовлетворен

Обнаружение и восстановление

Требования первого процесса удовлетворить невозможно из-за отсутствия доступного привода Blu-ray-дисков. Запрос второго процесса также нельзя удовлетворить, так как нет свободного сканера

К счастью, можно удовлетворить запрос третьего процесса, поэтому третий процесс запускается и в конечном итоге высвобождает все удерживавшиеся им ресурсы, в результате чего получается: $A = (2 \ 2 \ 2 \ 0)$

Обнаружение и восстановление

$$E = \begin{pmatrix} 4 & 2 & 3 & 1 \end{pmatrix}$$

Накопители на магнитной ленте
Плоттеры
Сканеры
Компакт-диски

$$A = \begin{pmatrix} 2 & 2 & 2 & 0 \end{pmatrix}$$

Накопители на магнитной ленте
Плоттеры
Сканеры
Компакт-диски

Матрица текущего
распределения

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Матрица запросов

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Обнаружение и восстановление

Теперь может быть запущен процесс 2, высвобождающий удерживаемые им ресурсы, в результате чего получается: $A = (4 \ 2 \ 2 \ 1)$

и может быть запущен оставшийся процесс

При этом взаимоблокировки в системе не возникает

Обнаружение и восстановление

Накопители на магнитной ленте
Плоттеры
Сканеры
Компакт-диски

$$E = (4 \quad 2 \quad 3 \quad 1)$$

Накопители на магнитной ленте
Плоттеры
Сканеры
Компакт-диски

$$A = (4 \quad 2 \quad 2 \quad 1)$$

Матрица текущего
распределения

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Матрица запросов

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}$$

Обнаружение и восстановление

Рассмотрим незначительное изменение ситуации.

Предположим, что процесс 3 нуждается в приводе Blu-ray-дисков, а также в двух ленточных накопителях и плоттере

Ни один из этих запросов не может быть удовлетворен, поэтому вся система в конечном итоге войдет в состояние взаимоблокировки

Даже если мы дадим процессу 3 два его ленточных накопителя и один плоттер, система войдет в состояние взаимоблокировки при запросе привода Blu-ray-дисков

Обнаружение и восстановление

Накопители на магнитной ленте
Плоттеры
Сканеры
Компакт-диски

$$E = (4 \quad 2 \quad 3 \quad 1)$$

Накопители на магнитной ленте
Плоттеры
Сканеры
Компакт-диски

$$A = (4 \quad 2 \quad 2 \quad 1)$$

Матрица текущего
распределения

$$C = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 2 & 0 & 0 & 1 \\ 0 & 1 & 2 & 0 \end{bmatrix}$$

Матрица запросов

$$R = \begin{bmatrix} 2 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 2 & 1 & 0 & 1 \end{bmatrix}$$


Обнаружение и восстановление

Теперь, когда мы знаем, как можно обнаружить взаимоблокировку (по крайней мере, при заранее известных статических запросах на выделение ресурсов), возникает вопрос, когда именно нужно приступить к их поиску

Можно, конечно, проводить проверку при выдаче каждого запроса на выделение ресурса. Тем самым будет обеспечено их обнаружение на самой ранней стадии, но это слишком обременительно для центрального процессора

Есть альтернативная стратегия, предусматривающая проверку каждые k минут или, может быть, только в том случае, когда степень загруженности процессора снижается относительно какого-то порога. Оценка загруженности процессора имеет определенный смысл, поскольку при участии во взаимоблокировке достаточного количества процессов работоспособными останутся лишь несколько процессов и центральный процессор будет часто простаивать

Обнаружение и восстановление

Предположим, что наш алгоритм обнаружения взаимоблокировки успешно отработал и обнаружил такую блокировку. Что же дальше?

Нужны какие-то методы выхода из нее, позволяющие системе восстановить работоспособность. Рассмотрим следующие:

- Восстановление за счет приоритетного овладения ресурсом
- Восстановление путем отката
- Восстановление путем уничтожения процессов

Ни один из них не обладает какой-то исключительной привлекательностью

Приоритетное овладение ресурсом

Иногда можно временно отобрать ресурс у его текущего владельца и передать его другому процессу. В большинстве случаев для этого может понадобиться вмешательство оператора, особенно в операционных системах пакетной обработки, запускаемых на универсальных машинах

Возможность отобрать ресурс у процесса, позволить использовать его другому процессу, а затем вернуть его без извещения процесса во многом зависит от природы этого ресурса

Восстановление этим способом зачастую затруднено или вовсе невозможно. Выбор процесса для приостановки обусловлен тем, какой именно процесс обладает тем ресурсом, который у него можно легко отобрать

Откат

Если разработчики системы и операторы вычислительной машины знают о том, что есть вероятность возникновения взаимоблокировки, они могут организовать периодическое создание процессами **контрольных точек**

Это означает, что состояние процесса записывается в файл, что позволит осуществить его последующий перезапуск.

Контрольные точки содержат не только образ памяти, но и состояние ресурсов, то есть информацию о том, какие ресурсы в данный момент выделены процессу

Для большей эффективности новая контрольная точка должна записываться не поверх старой, а в новый файл, чтобы во время выполнения процесса собралась целая последовательность контрольных точек

Откат

При обнаружении взаимоблокировки несложно определить, какие ресурсы нужны

Чтобы выйти из взаимоблокировки, процесс, владеющий необходимым ресурсом, откатывается назад к точке, предшествующей получению данного ресурса, для чего он запускается из одной из своих контрольных точек. Вся работа, выполненная после этой контрольной точки, теряется

Фактически процесс возвращается к предшествующему моменту, когда он еще не обладал тем ресурсом, который теперь выделен одному из участвующих во взаимоблокировке процессов. Если перезапущенный процесс пытается опять получить ресурс, ему приходится ждать, пока тот не станет доступен

Уничтожение процессов

Самым грубым, но и самым простым способом прервать взаимоблокировку является уничтожение одного или нескольких процессов

Можно уничтожить процесс, находящийся в цикле взаимоблокировки. Если повезет, то другие процессы смогут продолжить свою работу. Если это не поможет, то все можно повторить, пока цикл не будет разорван

В качестве альтернативы можно выбрать жертвой процесс, не находящийся в цикле, чтобы он высвободил удерживаемые им ресурсы. При этом подходе уничтожаемый процесс выбирается с особой тщательностью, потому что он должен удерживать ресурсы, необходимые некоторым процессам в цикле

Уничтожение процессов

По возможности лучше убить процесс, который может быть безболезненно перезапущен с самого начала. К примеру, компиляция всегда может быть перезапущена, поскольку все, что она делает, – это читает входной файл и создает объектный файл

Если процесс компиляции будет уничтожен на полпути, то первый запуск не повлияет на второй

А вот процесс, обновляющий базу данных, не всегда можно будет безопасно запустить во второй раз. Если процесс прибавляет единицу к какой-нибудь записи таблицы базы данных, то его первоначальный запуск, уничтожение, а затем повторный запуск приведут к неверному результату, поскольку к полю будет прибавлена двойка

Обнаружение и восстановление

При рассмотрении темы обнаружения взаимоблокировок мы предположили, что когда процесс запрашивает ресурсы, он просит их все сразу (матрица R)

Но в большинстве систем ресурсы запрашиваются по одному. Система должна уметь принимать решение, представляет выделение ресурса опасность или нет, и выделять его только в том случае, если это безопасно

В связи с этим возникает вопрос: существует ли алгоритм, который помог бы избежать взаимоблокировки, каждый раз делая правильный выбор?

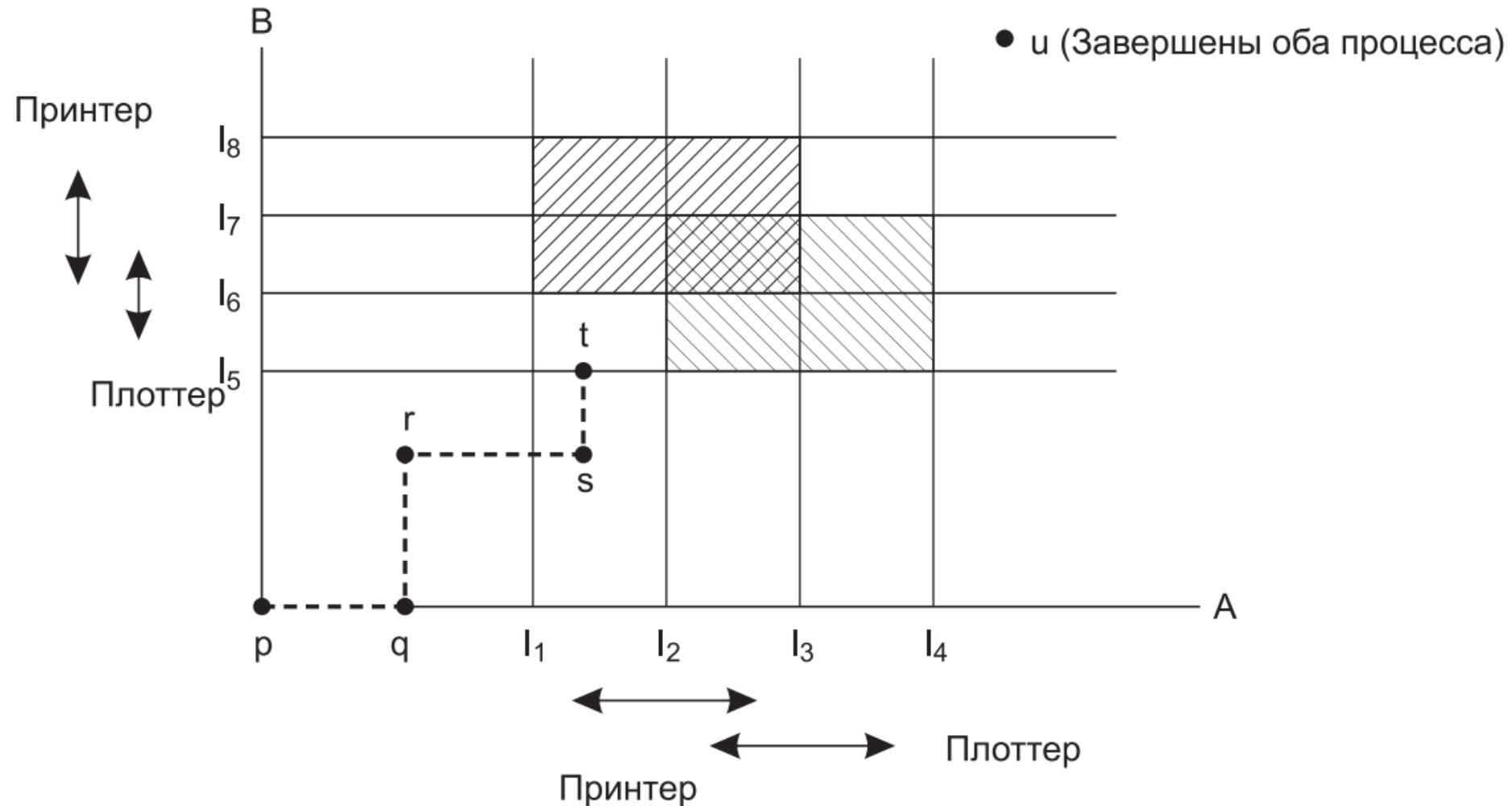
Ответом будет да, но при определенных условиях взаимоблокировки можно избежать, но только если заранее будет доступна вполне определенная информация

Безопасные состояния

Основные алгоритмы уклонения от взаимоблокировок основаны на концепции **безопасных состояний**. Перед тем как дать описание алгоритмов, нужно сделать небольшое отступление, чтобы рассмотреть концепцию безопасности в графическом, простом для понимания виде

Хотя графический подход не переводится непосредственно в пригодный к использованию алгоритм, он дает неплохое интуитивное понимание существа вопроса

Безопасные состояния



Безопасные состояния

На горизонтальной оси отображены номера команд, выполняемых процессом А

На вертикальной оси отображены номера команд, выполняемых процессом В

В команде I_1 процесс А запрашивает принтер, в команде I_2 он запрашивает плоттер. Принтер и плоттер высвобождаются командами I_3 и I_4 соответственно

Процессу В нужны плоттер с команды I_5 по команду I_7 и принтер с команды I_6 по команду I_8

Безопасные состояния

Каждая точка на графике представляет совместное состояние двух процессов. Изначально система находится в точке p , когда ни один процесс еще не выполнил ни одной инструкции

Если планировщик запустит процесс A первым, мы попадем в точку q , в которой процесс A выполнил какое-то количество команд, а процесс B еще ничего не сделал. В точке q траектория становится вертикальной, показывая, что планировщик решил запустить в работу процесс B

При наличии одного процессора все отрезки траектории могут быть только вертикальными или горизонтальными, но не диагональными. Кроме того, движение всегда происходит в северном или восточном направлении (вверх и вправо) и никогда не происходит в южном или западном (вниз и влево), поскольку процессы не могут работать, возвращаясь в прошлое

Безопасные состояния

Когда процесс А пересекает прямую I_1 на пути из r в s , он запрашивает, а затем получает в свое распоряжение принтер. Когда процесс В достигает точки t , он запрашивает плоттер

Особый интерес представляют заштрихованные области. Область со штриховкой из верхнего левого угла в правый нижний представляет промежуток времени, когда оба процесса удерживают принтер. Правило взаимного исключения делает попадание в эту область невозможным

Аналогично этому область, имеющая другую штриховку, представляет промежуток времени, когда оба процесса удерживают плоттер, и попадание в эту область также невозможно

Безопасные состояния

Если система войдет в прямоугольник, ограниченный по бокам прямыми I_1 и I_2 , а сверху и снизу прямыми I_5 и I_6 , то она в конце концов доберется до пересечения линий I_2 и I_6 и возникнет взаимоблокировка

В этот момент процесс А запрашивает плоттер, а процесс В запрашивает принтер, но оба ресурса будут уже выделены. Получается, что небезопасным является весь прямоугольник и в него входить нельзя

В точке t единственным безопасным действием будет работа процесса до тех пор, пока он не дойдет до команды I_4 . После нее для того, чтобы добраться до точки u , подойдет любая траектория

Безопасные состояния

Важный для понимания момент заключается в том, что в точке t процесс В запрашивает ресурс. Система должна принять решение, предоставлять его или нет

Если ресурс предоставляется, система попадает в небезопасную область и со временем входит в состояние взаимоблокировки

Чтобы предупредить взаимоблокировку, нужно приостановить процесс В до тех пор, пока процесс А не запросит и не высвободит плоттер

Безопасные состояния

При дальнейшем рассмотрении алгоритмов уклонения от взаимоблокировок используется информация, представленная ранее на слайде 51

В любой заданный момент времени существует текущее состояние, содержащее E, A, C и R. Состояние считается **безопасным**, если существует какой-то порядок планирования, при котором каждый процесс может доработать до конца, даже если все процессы внезапно и срочно запросят максимальное количество ресурсов

Это положение проще всего проиллюстрировать с помощью примера, в котором используется один ресурс

Безопасные состояния

	Имеет	Max
A	3	9
B	2	4
C	2	7

Свободно: 3

а

	Имеет	Max
A	3	9
B	4	4
C	2	7

Свободно: 1

б

	Имеет	Max
A	3	9
B	—	—
C	2	7

Свободно: 5

в

	Имеет	Max
A	3	9
B	—	—
C	7	7

Свободно: 0

г

	Имеет	Max
A	3	9
B	—	—
C	—	—

Свободно: 7

д

Безопасные состояния

Теперь предположим, что исходное состояние системы (а) остаётся таким же, но в данный момент процесс А запрашивает и получает еще один ресурс и система переходит в состояние (б). Сможем ли мы найти последовательность, которая гарантирует безопасную работу системы?

Безопасные состояния

	Имеет	Max
A	3	9
B	2	4
C	2	7

Свободно: 3

а

	Имеет	Max
A	4	9
B	2	4
C	2	7

Свободно: 2

б

	Имеет	Max
A	4	9
B	4	4
C	2	7

Свободно: 0

в

	Имеет	Max
A	4	9
B	—	—
C	2	7

Свободно: 4

г

Безопасные состояния

Следует отметить, что небезопасное состояние само по себе не является состоянием взаимоблокировки. Начиная с состояния (б), система может поработать некоторое время

Фактически может даже успешно завершиться работа одного из процессов. Кроме того, процесс А может высвободить один ресурс еще до запроса дополнительного ресурса, позволяя успешно завершить работу процессу С, а системе в целом избежать взаимоблокировки

Таким образом, разница между безопасным и небезопасным состоянием заключается в том, что в безопасном состоянии система может гарантировать, что все процессы закончат свою работу, а в небезопасном состоянии такой гарантии дать нельзя

Алгоритм банкира

Алгоритм планирования, позволяющий избегать взаимоблокировок, был разработан Дейкстрой и известен как **алгоритм банкира**

Он представляет собой расширение алгоритма который мы рассматривали для обнаружения взаимоблокировки при использовании одного ресурса каждого типа

Модель алгоритма основана на примере банкира маленького городка, имеющего дело с группой клиентов, которым он выдал ряд кредитов. (Много лет назад банки не давали кредиты, пока не убеждались в том, что они могут быть возвращены)

Алгоритм проверяет, ведет ли выполнение каждого запроса к небезопасному состоянию. Если да, то запрос отклоняется. Если удовлетворение запроса к ресурсу приводит к безопасному состоянию, ресурс предоставляется процессу

Алгоритм банкира

Допустим есть четыре клиента: А, В, С и D, каждый из которых получил определенное количество единиц кредита (например, 1 единица равна 1000 долларов)

Банкир знает, что не всем клиентам тотчас же понадобится максимальная сумма их кредита, поэтому для обслуживания их потребностей он зарезервировал только 10 единиц, а не все 22, которые нужны клиентам

Чтобы провести аналогию с компьютерной системой, будем считать, что клиенты – это процессы, единицы – накопители на магнитной ленте, а банкир – операционная система

Алгоритм банкира

	Имеет	Max
A	0	6
B	0	5
C	0	4
D	0	7

Свободно: 10

а

	Имеет	Max
A	1	6
B	1	5
C	2	4
D	4	7

Свободно: 2

б

	Имеет	Max
A	1	6
B	2	5
C	2	4
D	4	7

Свободно: 1

в

Алгоритм банкира

Клиенты занимаются своими делами, время от времени запрашивая ссуды (то есть запрашивая ресурсы). В какой-то определенный момент возникает ситуация (б)

Это состояние не представляет опасности, поскольку при оставшихся двух единицах банкир может отложить выполнение любых запросов, за исключением запроса клиента С, позволяя С завершить свои дела и высвободить все четыре своих ресурса

Имея в своем распоряжении четыре единицы ресурса, банкир может позволить получить необходимые единицы либо D, либо В и т. д.

Алгоритм банкира

Рассмотрим, что получится, если запрос от В одной дополнительной единицы будет удовлетворен в ситуации (б):

Мы получим небезопасную ситуацию (в)

Если все клиенты внезапно запросят максимальные ссуды, банкир не сможет удовлетворить никого из них и мы получим взаимоблокировку

Небезопасное состояние необязательно приводит к взаимоблокировке, поскольку клиенту может и не понадобиться максимальная сумма кредита, но банкир не может рассчитывать на это

Алгоритм банкира

Алгоритм банкира рассматривает каждый запрос по мере поступления и проверяет, приведет ли его удовлетворение к безопасному состоянию: если да, то запрос удовлетворяется, в противном случае запрос откладывается до лучших времен

Чтобы понять, является ли состояние безопасным, банкир проверяет, может ли он предоставить достаточно ресурсов для удовлетворения запросов какого-нибудь клиента

Если да, то эти ссуды считаются возвращенными, после чего проверяется следующий ближайший к пределу займа клиент и т. д. Если в конечном счете все ссуды могут быть погашены, состояние является безопасным и исходный запрос можно удовлетворить

Тупики и подходы к их разрешению

Как все-таки можно избежать взаимоблокировок в реальных системах?

Ведь мы уже убедились в том, что уклониться от них по сути невозможно, поскольку для этого требуется информация о будущих запросах, о которых ничего не известно

Чтобы ответить на этот вопрос, вернемся назад к четырем условиям, сформулированным Коффманом и посмотрим, смогут ли они дать нам ключ к решению проблемы

Если мы сможем гарантировать, что хотя бы одно из этих условий никогда не будет выполнено, то взаимоблокировки станут структурно невозможными

Атака условия взаимного исключения

Сначала предпримем атаку на условие взаимного исключения. Если в системе нет ресурсов, отданных в единоличное пользование одному процессу, мы никогда не попадем в ситуацию взаимоблокировки

Данные проще всего сделать доступными только для чтения, чтобы процессы могли их использовать одновременно. Но также понятно, что если позволить двум процессам одновременно печатать данные на принтере, то это приведет к хаосу. За счет использования очереди на печать (спулинга) выдавать свои выходные данные могут сразу несколько процессов

В этой модели единственным процессом, который фактически запрашивает физический принтер, является демон принтера. Так как демон не запрашивает никакие другие ресурсы, взаимоблокировки, связанные с принтером, можно исключить

Если демон запрограммирован на начало печати еще до того, как все выходные данные попали в очередь на печать, принтер может простоять впустую, если выводящий данные процесс решит подождать несколько часов после первого пакета выходных данных

Атака условия взаимного исключения

Поэтому демоны обычно программируются так, чтобы начинать печать, только если доступен полный файл выходных данных. Но само по себе это решение может привести к взаимоблокировке

Что получится, если каждый из двух процессов заполнит по половине доступного пространства, выделенного на диске под очередь на печать своими выходными данными, и ни один из них не сформирует свои полные выходные данные?

В таком случае мы получим два процесса, завершивших формирование только части, но не всего объема своих выходных данных, и не имеющих возможности продолжить свою работу. Ни один из процессов не сможет когда-либо завершиться, и мы получим взаимоблокировку, связанную с выводом данных на диск

И все же здесь проглядывается намек на идею, которая довольно часто применяется на практике. Следует избегать выделения ресурса, если в нем нет насущной потребности, и постараться, чтобы как можно меньше процессов могло фактически требовать получения этого ресурса

Атака условия удержания и ожидания

Второе из условий, сформулированных Коффманом, выглядит несколько более обещающим

Если можно будет помешать процессам, удерживающим ресурсы, войти в фазу ожидания дополнительных ресурсов, то можно будет исключить и взаимоблокировку

Один из способов достижения этой цели заключается в том, чтобы заставить все процессы запрашивать все свои ресурсы до начала выполнения своей работы

Если все доступно, то процессу будет выделено все, что ему требуется, и он сможет доработать до завершения. Если один или несколько ресурсов заняты, ничего не будет выделяться и процесс будет просто ждать

Атака условия удержания и ожидания

Проблема, непосредственно связанная с этим подходом, состоит в том, что многие процессы не знают, сколько ресурсов им понадобится, пока не начнут работу. Фактически если бы они об этом знали, то можно было бы использовать и алгоритм банкира

Вторая проблема состоит в том, что при таком подходе ресурсы не будут использоваться оптимально

Возьмем, к примеру, процесс, считывающий данные с входной ленты, анализирующий их в течение часа, а затем записывающий выходную ленту, да еще и выводящий результаты на плоттер. Если все ресурсы должны быть запрошены заранее, то процессор на целый час займет выходной накопитель на магнитной ленте и плоттер

Атака условия удержания и ожидания

И все-таки некоторые пакетные системы на универсальных машинах требуют, чтобы пользователи перечисляли все ресурсы в первой строке каждого задания. Затем система немедленно заранее распределяет все ресурсы и удерживает их до тех пор, пока они станут не нужны заданию (или в простейшем случае, пока выполнение задания не будет завершено)

Несмотря на то что этот метод обременяет программиста и расточительно расходует ресурсы, он предотвращает возникновение взаимоблокировок

Слегка отличающийся метод нарушения условия удержания и ожидания заключается в требовании от процесса, запрашивающего ресурс, вначале временно высвободить все ресурсы, удерживаемые им на данный момент. Затем этот процесс пытается заполучить сразу все, что ему требуется

Атака условия невыгружаемости

Возможна также атака и третьего условия (невыгружаемости).

Если процессу выделен принтер и он распечатал лишь половину своих выходных данных, то принудительно отобрать у него принтер по причине недоступности запрошенного плоттера в лучшем случае будет слишком затруднительно, а в худшем – просто невозможно

Тем не менее, чтобы избежать подобной ситуации, некоторые ресурсы могут быть виртуализированы. Сохранение очереди на печать на диске и предоставление возможности доступа к реальному принтеру только демону принтера исключает возникновение взаимоблокировок с участием принтера, хотя и создает одну из таких потенциальных возможностей в отношении дискового пространства. Но при наличии дисков большой емкости исчерпание дискового пространства становится маловероятным

Атака условия невыгружаемости

Однако не все ресурсы могут быть виртуализированы подобным образом. К примеру, чтобы записи в базах данных или в таблицах внутри операционной системы могли использоваться, они должны быть заблокированы, и здесь закладывается потенциальная вероятность взаимоблокировки

Атака условия циклического ожидания

Осталось только одно условие. Циклическое ожидание можно устранить несколькими способами

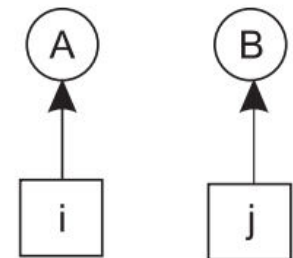
Один из них заключается в простом выполнении правила, которое гласит, что процессу в любой момент времени дано право только на один ресурс. Если нужен второй ресурс, процесс обязан освободить первый. Но подобное ограничение неприемлемо для процесса, копирующего огромный файл с магнитной ленты на принтер

Атака условия циклического ожидания

Другой способ, позволяющий избежать циклического ожидания, заключается в поддержке общей нумерации всех ресурсов

Теперь действует следующее правило: процессы могут запрашивать ресурс, когда только пожелают, но все запросы должны быть сделаны в порядке нумерации ресурсов. Процесс может запросить сначала принтер, затем накопитель на магнитной ленте, но не может сначала потребовать плоттер, а затем принтер

1. Принтер
2. Сканер
3. Плоттер
4. Накопитель на магнитной ленте
5. Устройство для чтения компакт-дисков



Атака условия циклического ожидания

Если придерживаться этого правила, то у графа распределения ресурсов никогда не будет циклов

Посмотрим, почему это имеет место в случае двух процессов, показанных ранее (б)

Взаимоблокировка может произойти, только если процесс А запросит ресурс j , а процесс В запросит ресурс i

Предположим, что ресурсы i и j относятся к разным типам, тогда они будут иметь и разные номера. Если $i > j$, то процессу А не разрешается запрашивать ресурс j , потому что его номер меньше, чем номер уже имеющегося у него ресурса. Если же $i < j$, то процесс В не может запрашивать ресурс i , потому что его номер меньше номера уже удерживаемого этим процессом ресурса

Так или иначе, взаимоблокировка невозможна

Атака условия циклического ожидания

При работе более чем с двумя процессами сохраняется та же самая логика. В любой момент времени один из предоставленных ресурсов будет иметь наивысший номер

Процесс, использующий этот ресурс, никогда не запросит тот ресурс, который уже распределен. Он или закончит свою работу, или, в худшем случае, запросит ресурс с еще большим номером, а все такие ресурсы доступны. В итоге процесс завершит работу и высвободит свои ресурсы

К этому времени какой-нибудь другой процесс будет удерживать ресурс с самым большим номером и тоже сможет завершить свою работу. Короче говоря, существует сценарий, по которому все процессы завершают свою работу, поэтому никаких взаимоблокировок и не возникает

Атака условия циклического ожидания

При незначительном изменении этого алгоритма исключается требование приобретения ресурсов в строго возрастающем порядке и просто требуется, чтобы ни один процесс не запрашивал ресурс с меньшим номером, чем номер того ресурса, который он уже удерживает

Если процесс сначала запрашивает ресурсы 9 и 10, а затем высвобождает их обоих, то это во всех отношениях равнозначно новому началу работы, поэтому теперь уже незачем запрещать ему запрос ресурса 1

Хотя порядковая нумерация ресурсов исключает проблему взаимоблокировок, может не представиться возможности подобрать порядок, удовлетворяющий абсолютно всех. Когда ресурсы включают в себя элементы таблицы процессов, дисковое пространство очереди печати, заблокированные записи базы данных и другие абстрактные ресурсы, количество потенциальных ресурсов и различных применений может быть настолько большим, что не сможет работать никакое упорядочение

Тупики и подходы к их разрешению

Различные методы предупреждения взаимоблокировок сведены в табличку

Условие	Метод
Взаимное исключение	Организация очереди на диске
Удержание и ожидание	Изначальный запрос всех ресурсов
Невыгружаемость	Отобрать ресурсы
Циклическое ожидание	Провести порядковую нумерацию ресурсов

Операционные системы

Лекция 9

Тупики